Leghosszabb növekvő részsorozat

Algoritmus dinamikus programozással, összefüggések és alkalmazások

Szerző: Nikházy László
Előadó: Varga Péter

2026. január 23.

Leghosszabb növekvő részsorozat

Adott egy sorozat: $x_1, x_2, \ldots, x_n$

Definíció: Egy $i_1 < i_2 < \cdots < i_k$ indexsorozat *növekvő részsorozat*, ha $x_{i_1} < x_{i_2} < \cdots < x_{i_k}$.

Feladat (LIS, Longest Increasing Subsequence): Határozzuk meg a **leghosszabb szigorúan növekvő részsorozat** hosszát (és később magát a részsorozatot is).

Megjegyzés:

- A részsorozat nem feltétlenül összefüggő. Az eredeti sorozatból elemek elhagyásával kapjuk.
- Az előadás során végig **szigorúan növekvő** LIS-sel foglalkozunk, de teljesen analóg módon működne minden, ha megengednénk az egyenlőséget.

Példa:

Sorozat: 3 1 5 2 6 4 9

Egy leghosszabb növekvő részsorozat: 1 2 4 9 (hossz: 4)

LIS – $O(n^2)$ dinamikus programozás

Részprobléma:

Legyen

$$\texttt{dp[i]} = \text{a } x_i\text{-ben végződő növekvő részsorozat maximális hossza}.$$

Átmenet:

$$\texttt{dp[i]} = 1 + \max\{\texttt{dp[j]} \mid j < i \text{ és } x_j < x_i\}.$$

Ha nincs ilyen j, akkor $\texttt{dp[i]} = 1$.

Válasz:

$$\max_{1 \le i \le n} \texttt{dp[i]}.$$

Időkomplexitás:

$$O(n^2)$$

LIS – Rekonstrukció ($O(n^2)$)

A részsorozat visszaállításához tároljuk:

- `prev[i]`: annak az indexe, amely közvetlenül megelőzi x_i-t egy optimális növekvő részsorozatban.

Átmenet frissítésekor:

- ha `dp[j]` + 1 > `dp[i]`, akkor
  - `dp[i]` = `dp[j]` + 1,
  - `prev[i]` = `j`.

Rekonstrukció menete:

- Keressük meg azt az `i` indexet, ahol `dp[i]` maximális.
- A `prev` mutatók mentén visszafelé haladva megkapjuk a LIS elemeit fordított sorrendben.

Megjegyzés: Ez a módszer nemcsak a hosszát, hanem magát a leghosszabb növekvő részsorozatot is előállítja.

Lehet-e gyorsabban?

Megjegyzés:

- A klasszikus DP reláció

$$\texttt{dp[i]} = 1 + \max\{\texttt{dp[j]} \mid \texttt{j} < \texttt{i},\ a_j < a_i\}$$

  szegmensfával vagy Fenwick-fával $\texttt{O(N log N)}$-re gyorsítható.
- Ezt a megközelítést most **nem fejtjük ki**, csak megemlítjük mint alternatívát.

Ehelyett: bemutatunk egy másik, sokkal elegánsabb $\texttt{O(N log N)}$-es DP-megközelítést.

Másik DP megfogalmazás

Definiáljuk a következőt:

`dp[k]`: a lehető **legkisebb végződésű szigorúan növekvő** részsorozat végső eleme, amelynek hossza pontosan `k`.

Példa:

$$a = (3,\ 1,\ 5,\ 2,\ 6)$$

- $dp[1] = 1$ (pl. (1))
- $dp[2] = 2$ (pl. $(1, 2)$)
- $dp[3] = 6$ (pl. $(1, 2, 6)$)

Megfigyelés:

$$dp[1] < dp[2] < dp[3] < \ldots$$

tehát a `dp` tömb **szigorúan növekvő**.

DP frissítése egy új elemnél

Legyen az aktuális elem $x = a_i$.

A kérdés:

Melyik a legnagyobb `k`*, amelyre* `dp[k] < x`*?*

Ekkor:

$$\texttt{dp[k+1]} = \texttt{min(dp[k+1], x)}$$

Miért jó ez?

- A `dp` tömb mindig a „legígéretesebb” végződéseket tartja nyilván.
- Tulajdonképpen most is minden elemnél meghatározzuk, hogy milyen hosszú az ott végződő leghosszabb növekvő részsorozat, amikor a legnagyobb olyan `k`-t keressük, melyre `dp[k] < x`.

Hatékony megvalósítás:

- A `dp` tömb növekvő.
- Bináris kereséssel keressük az első `dp[k]` $\geq$ `x` pozíciót.

LIS — A dp tömb alakulása egy példán

Sorozat: `(3, 1, 5, 2, 6, 4, 9)`

Lépés	Aktuális elem	`dp` állapota
1	3	$\boxed{3}$

LIS — A dp tömb alakulása egy példán

Sorozat: `(3, 1, 5, 2, 6, 4, 9)`

Lépés	Aktuális elem	`dp` állapota
1	3	$\boxed{3}$
2	1	$\boxed{1}$

LIS — A dp tömb alakulása egy példán

Sorozat: `(3, 1, 5, 2, 6, 4, 9)`

Lépés	Aktuális elem	`dp` állapota
1	3	$\boxed{3}$
2	1	$\boxed{1}$
3	5	$\boxed{1}\boxed{5}$

LIS — A dp tömb alakulása egy példán

Sorozat: `(3, 1, 5, 2, 6, 4, 9)`

Lépés	Aktuális elem	`dp` állapota
1	3	3
2	1	1
3	5	1 5
4	2	1 2

LIS — A dp tömb alakulása egy példán

Sorozat: `(3, 1, 5, 2, 6, 4, 9)`

Lépés	Aktuális elem	`dp` állapota
1	3	3
2	1	1
3	5	1 5
4	2	1 2
5	6	1 2 6

LIS — A dp tömb alakulása egy példán

Sorozat: `(3, 1, 5, 2, 6, 4, 9)`

Lépés	Aktuális elem	`dp` állapota
1	3	3
2	1	1
3	5	1 5
4	2	1 2
5	6	1 2 6
6	4	1 2 4

LIS — A dp tömb alakulása egy példán

Sorozat: `(3, 1, 5, 2, 6, 4, 9)`

Lépés	Aktuális elem	`dp` állapota
1	3	3
2	1	1
3	5	1 5
4	2	1 2
5	6	1 2 6
6	4	1 2 4
7	9	1 2 4 9

A végén: $|\mathtt{dp}| = 4$, tehát a LIS hossza 4.

LIS – $O(N \log N)$ algoritmus

Algoritmus:

- Kezdetben `dp` üres.
- Balról jobbra feldolgozzuk az elemeket.
- Minden $x = a_i$ esetén:
  - Keressük meg az első `k` indexet, amelyre $dp[k] \geq x$.
  - Ha nincs ilyen, `dp` végére tesszük `x`-et.
  - Különben $dp[k] = x$.

Időkomplexitás: $O(N \log N)$

Fontos:

- A `dp` tömb **nem feltétlenül** egy konkrét LIS.
- A hossza viszont pontosan a LIS hossza.

LIS – Alap kód (csak hossz)

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      int n; cin >> n;
6      vector<int> a(n);
7      for (int i = 0; i < n; i++) cin >> a[i];
8
9      vector<int> dp;
10     for (int x : a) {
11         auto it = lower_bound(dp.begin(), dp.end(), x);
12         if (it == dp.end()) dp.push_back(x);
13         else *it = x;
14     }
15
16     cout << dp.size() << "\n";
17 }
```

Hogyan rekonstruáljuk a LIS-t?

Szeretnénkegy konkrét növekvő részsorozatot is.

Ehhez extra információkat tárolunk:

- **pos[k]**: melyik indexen végződik a k hosszú részsorozat.
- **parent[i]**: az a_i előtti elem indexe a LIS-ben.

Ötlet:

- Amikor $x = a_i$ bekerül a k-adik pozícióra, akkor

$$\texttt{parent[i]} = \texttt{pos[k} - 1\texttt{]}.$$

- Frissítjük: $\texttt{pos[k]} = \texttt{i}$.

A végén visszafelé haladva rekonstruálható a teljes LIS.

LIS – Kód rekonstrukcióval

```
int n; cin >> n;
vector<int> a(n);
for (int i = 0; i < n; i++) cin >> a[i];
vector<int> dp, pos, parent(n, -1);
for (int i = 0; i < n; i++) {
    int k = lower_bound(dp.begin(), dp.end(), a[i]) - dp.begin();
    if (k == dp.size()) {
        dp.push_back(a[i]);
        pos.push_back(i);
    } else {
        dp[k] = a[i];
        pos[k] = i;
    }
    if (k > 0) parent[i] = pos[k - 1];
}
vector<int> lis;
int cur = pos.back();
while (cur != -1) {
    lis.push_back(a[cur]);
    cur = parent[cur];
}
reverse(lis.begin(), lis.end());
cout << lis.size() << "\n";
for (int x : lis) cout << x << " ";
```

Láttuk már ezt az algoritmust korábban?

Melyik feladatnál?

Láttuk már ezt az algoritmust korábban?

Melyik feladatnál?
Igen! Pontosan ugyanez az algoritmus megjelent már a **CSES Towers** feladatnál a mohó témakörben.

Összehasonlítás:

- LIS esetén:
  - `dp[k]` = egy `k` hosszú növekvő részsorozat minimális végződése
  - frissítés: `lower_bound`
- Towers esetén:
  - a tornyok tetejeinek értékeit tartjuk nyilván
  - frissítés: `upper_bound`

Megjegyzés: Ha a Towers feladatban megengedjük, hogy **egyenlő méretű kockák egymásra kerüljenek**, akkor a két algoritmus *pontosan ugyanaz*.

DP vagy mohó algoritmus?

Felmerül a kérdés:

> *Hogyan lehet, hogy ugyanaz a kód egyszer dinamikus programozás, máskor pedig mohó algoritmus?*

Válasz: Attól függ, hogyan *értelmezzük* az állapotokat.

- Tekinthetjük úgy, hogy:
  - `dp[k]` egy jól definiált DP-állapot,
  - és minden elemnél ezeket az állapotokat frissítjük.
- De úgy is, hogy:
  - minden lépésben egy lokálisan legjobb döntést hozunk,
  - ami a jövőre nézve a lehető legkedvezőbb állapotot hagyja meg.

A DP-állapot frissítése mögött egy **mohó elv** áll:

> *minden hosszra a lehető legkisebb végződést tartjuk meg.*

LIS és tornyok — egy szép dualitás

A kapcsolat még erősebb, az algoritmus helyességének bizonyítását kaphatjuk belőle:

- Vegyünk egy tetszőleges **növekvő részsorozatot**.
- A tornyokba pakoláskor ezeknek az elemeknek **mind külön toronyba kell kerülniük**.

Tehát:

- bármely növekvő részsorozat hossza $\leq$ tornyok száma.

Van egy algoritmus, ami egyszerre ad meg:

- egy $\mathtt{K}$ hosszú növekvő részsorozatot,
- és egy $\mathtt{K}$ toronyból álló felbontást.

Ebből a tényből és a fenti egyszerűen belátható egyenlőtlenségből már következik, hogy az algoritmus által megadott növekvő részsorozatnál nincs hosszabb, és nem lehet kevesebb toronyba pakolni az elemeket.

Következmény: LIS hossza = Tornyok minimális száma.

A két feladat közötti kapcsolat talán kevésbé meglepő, ha átfogalmazzuk a tornyos feladatot.

Tornyos feladat másik megfogalmazása: Adott egy sorozat. Bontsuk fel minimális számú monoton csökkenő részsorozatra.

Variációk

Kockákból legmagasabb torony (Mester)

Adott $\mathtt{N}$ ($1 \leq \mathtt{N} \leq 100\,000$) építőkocka, mindegyikhez két adat tartozik:

- oldalhossz ($1 \leq \mathtt{S_i} \leq 10^9$),
- súly ($1 \leq \mathtt{W_i} \leq 10^9$).

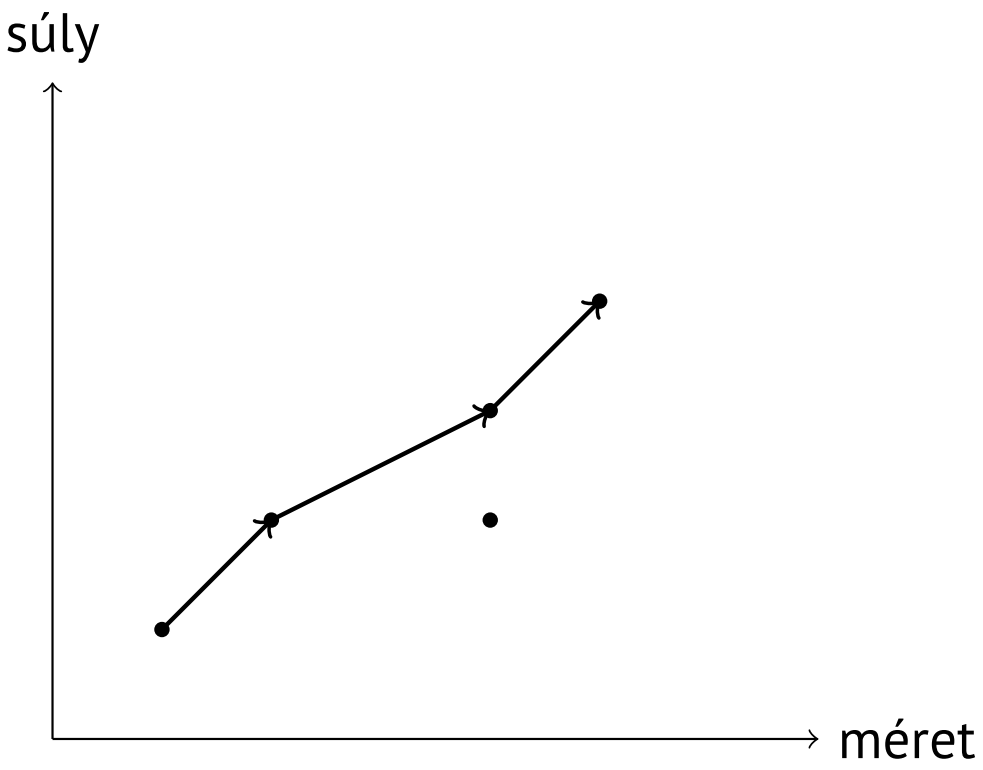
Egy kocka csak akkor kerülhet egy másikra, ha **mindkét paramétere szigorúan kisebb**.

Feladat:

- Határozzuk meg a megépíthető legtöbb kockából álló tornyot!
- Adjunk meg egy konkrét optimális építési sorrendet is!

Kockákból legmagasabb torony - Szemléltetés

Kockák (méret, súly):



Egy lehetséges növekvő lánc:

$$(1, 1) \to (2, 2) \to (4, 3) \to (5, 4)$$

Ez pontosan egy stabil toronynak felel meg alulról felfelé.

Kockákból legmagasabb torony - Kulcsötlet

Minden kockát tekintsünk egy számpárnak:

$$(\text{méret},\ \text{súly}) \rightarrow \text{egy pont a síkon}$$

A feladat lényege:

> *válasszunk ki és rendezzünk sorba minél több pontot úgy, hogy mindkét koordináta szigorúan növekvő legyen.*

Ez tulajdonképpen:

- **leghosszabb szigorúan növekvő sorozat**
- kétdimenziós pontok *halmazán* belül.

Kulcsötlet:

- rendezzük a kockákat az egyik koordináta szerint,
- a másik koordinátában keressünk LIS-t.

Így a probléma visszavezethető **egydimenziós LIS-re**.

Kockákból legmagasabb torony - Egyenlő elemek

Tegyük fel, hogy a kockákat **méret szerint növekvő sorrendbe rendezzük**.

Ezután a súlyokon keresünk LIS-t.

Probléma: Mi történik, ha két kockának azonos a mérete?

Megoldás:

- Ha az első koordináta egyenlő, akkor a második szerint **csökkenő sorrendbe** rendezzük őket.

Miért jó ez?

- Így két azonos méretű kocka nem kerülhet bele ugyanabba a szigorúan növekvő LIS-be.
- Megakadályozzuk a hibás „egymásra rakást”.

Ez egy klasszikus technika **többdimenziós LIS-feladatoknál**.

Hadjárat (Válogató 2020)

A hódító királynő az útvonal mentén sorban halad $\mathtt{N}$ ($1 \leq \mathtt{N} \leq 100\,000$) városon keresztül. Minden városra adott:

- $\mathtt{R_i}$: rabszolgák száma ($1 \leq \mathtt{R_i} \leq 1\,000\,000$),
- $\mathtt{A_i}$: arany mennyisége ($1 \leq \mathtt{A_i} \leq 1\,000\,000$).

Ha elfoglalja az $\mathtt{i_1} < \mathtt{i_2} < \cdots < \mathtt{i_k}$ városokat, akkor igaznak kell lennie:

$$\mathtt{R_{i_1}} < \mathtt{R_{i_2}} < \cdots < \mathtt{R_{i_k}} \quad \text{és} \quad \mathtt{A_{i_1}} < \mathtt{A_{i_2}} < \cdots < \mathtt{A_{i_k}}.$$

Feladat:

- Határozzuk meg a maximális $\mathtt{k}$-t!
- Adjunk meg egy ilyen várossorrendet is!

Hadjárat - Geometriai értelmezés

Tekintsük a városokat pontoknak a síkon: (R_i, A_i)

Ekkor a feladat:

> *Keressünk egy leghosszabb olyan részsorozatot, amelyben a pontoknak index szerint növekvő sorrendben mindkét koordinátája szigorúan növekszik.*

Ez egy **LIS-probléma** számpárokon.

Hangsúlyozzuk a különbséget a korábbi feladathoz képest:

Építőkockás feladatban

- számpárok halmazából választunk tetszőleges sorrendben;
- ha ez egyik koordináta szerint rendezzük, akkor el is hagyhatjuk azt, és visszavezetjük az egydimenziós LIS feladatra.

Hadjárat feladatban

- számpárok sorozatából választunk részsorozatot, tehát kötött a sorrend;
- gondolhatunk úgy is rá, hogy az index a harmadik koordináta, ami szerint már rendezve vannak.

Hadjárat - $O(N^2)$ megoldás

$O(N^2)$ megoldás van a LIS $O(N^2)$ DP megoldásával analóg módon.

`DP[i]` legyen az `i`-edik városban végződő leghosszabb hadjárat hossza.

A kiszámítása:

$\texttt{DP[i]} = \max\{\texttt{DP[j]}+1$, ahol $1 \leq \texttt{j} < \texttt{i}$ és $\texttt{R[j]} < \texttt{R[i]}$ és $\texttt{A[j]} < \texttt{A[i]}\}$

A megoldás előállítása is megy a korábban látott módon:

- minden elemhez eltároljuk `prev[i]`-t,
- ami annak az indexe, amely közvetlenül megelőzi `i`-t egy optimális növekvő részsorozatban.

Viszont ez a megoldás az időlimit miatt csak részpontokat ér, hatékonyabb megoldás kell.

`O(N log N)` algoritmus nem működik?

Probléma:

- Két pont lehet úgy „összehasonlíthatatlan”, hogy egyik sem kisebb a másiknál.
- Ilyenkor nem elég egyetlen „legjobb végződést” eltárolni, amit az `O(N log N)`-es LIS algoritmus csinálna.

Vegyük a következő pontokat (index szerinti sorrendben):

`(1, 1), (4, 2), (2, 5), (3, 6)`

- `(4, 2)`-ben és `(2, 5)`-ben is végződik egy 2 hosszú növekvő részsorozat.
- De egyik sem „jobb” a másiknál: az egyiknek nagyobb az `x`, a másiknak nagyobb az `y` koordinátája.
- Tehát nem választhatjuk ki közülük önkényesen az egyiket.
- Mindkettőt meg kell őriznünk mint lehetséges végződést.

Ez az oka annak, hogy a klasszikus `dp[k] = minimális végződés` megközelítést át kell dolgozni.

Hadjárat megoldás — Haladó LIS kétdimenzióban

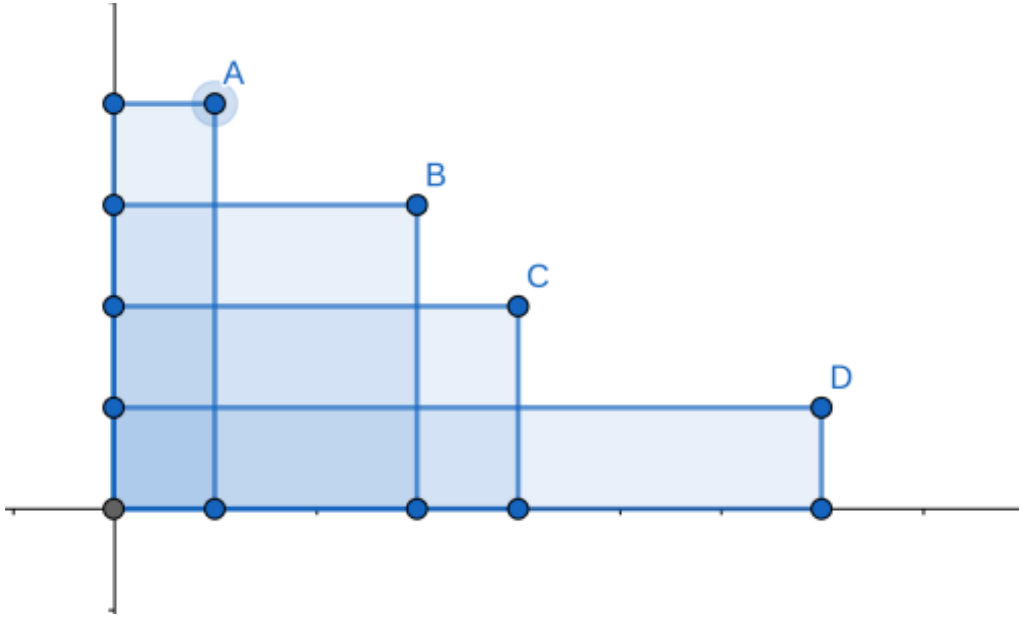
Általánosítjuk a klasszikus LIS-algoritmust.

Definíció:

- `best[k]`: azon pontok **halmaza**, amelyekben egy `k` hosszú hadjárat végződhet, és egyik sem dominálja a másikat.

Tulajdonság:

- Egy `best[k]` halmazban a pontok `x` szerint növekvők,
- `y` szerint szigorúan csökkenők.

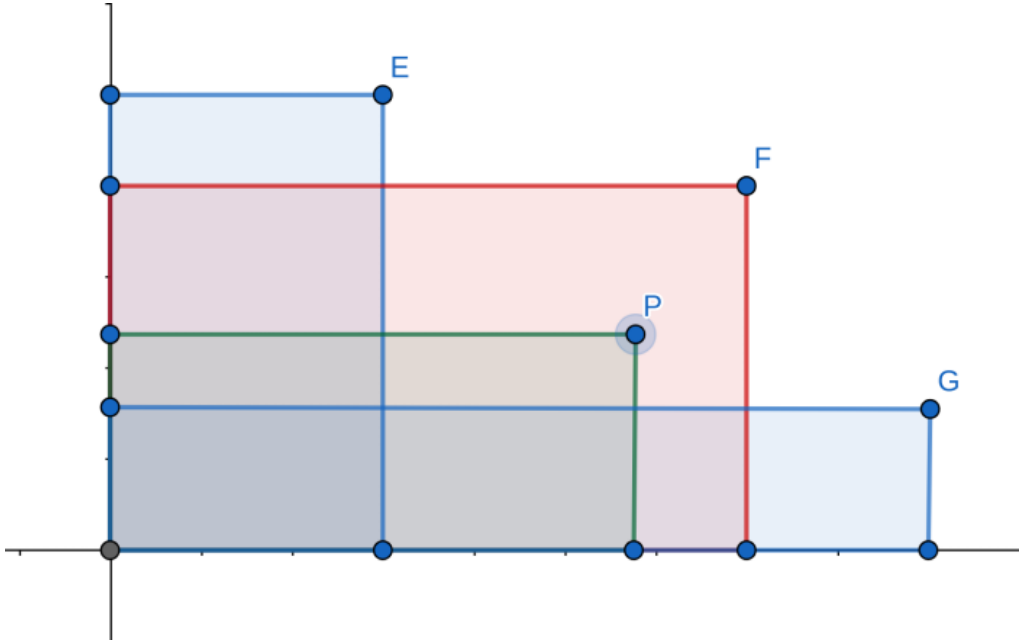


Hadjárat megoldás — Haladó LIS kétdimenzióban

Egy új `P` pont feldolgozása:

- Binárisan megkeressük a legnagyobb `k`-t, amelyre `P` valamely `best[k]`-beli pont mögé tehető.
- Ekkor `P` egy `k` + 1 hosszú sorozat végére kerül.
- Beszúrjuk `P`-t `best[k + 1]`-be, és töröljük az általa dominált pontokat.

Az utolsó lépés szemléltetve:

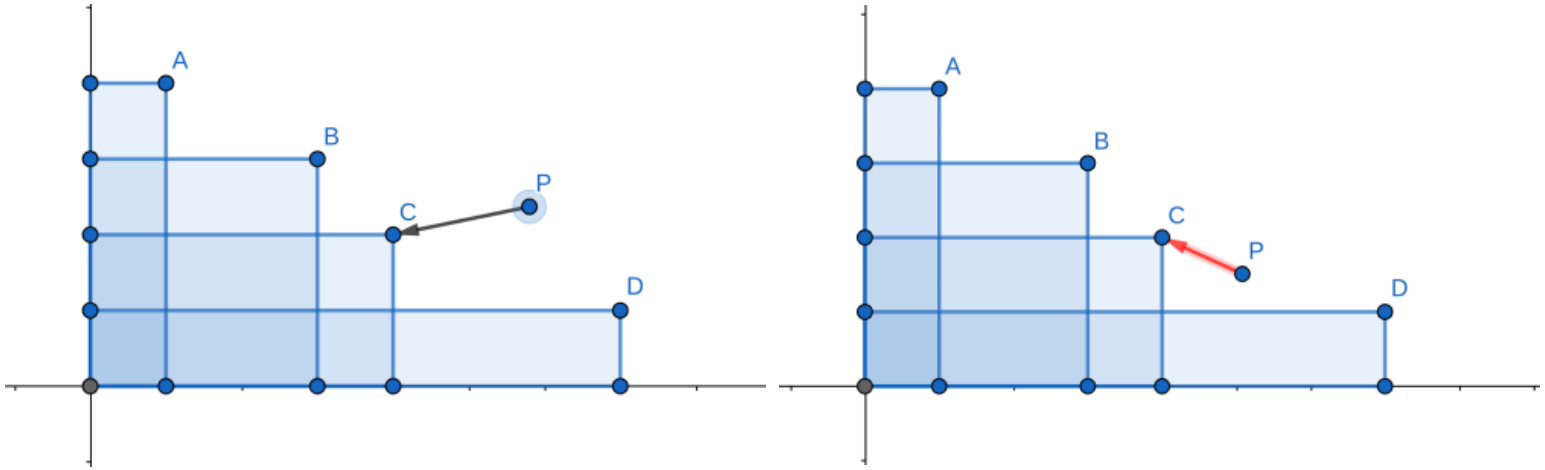


Hadjárat megoldás — Megvalósítás részletei

Amikor binárisan keressük a legnagyobb `k`-t egy `P` ponthoz, akkor el kell tudnunk dönteni, hogy `P` valamely `best[k]`-beli pont mögé tehető-e.

Megvalósítás:

- `best[k]` legyen egy `set` a pontok első koordinátája szerint rendezve
- így gyorsan meg tudjuk mondani, hogy egy újabb pontot (P) ezek egyike mögé lehet-e tenni:
  - megkeressük az utolsót, aminek az x koordinátája kisebb
  - és megnézzük, hogyan viszonyul ehhez a P y koordinátája.



Komplexitás: $O(N \log^2 N)$

Hadjárat - Kód

```
1  struct pont {
2    int x, y, id;
3    bool operator<(const pont& p) const {
4      return x < p.x || x == p.x && y < p.y;
5    }
6  };
7  // best[i]: i hosszú sorozatok legjobb végződései
8  vector<set<pont>> best(1, set<pont>({{-1, -1, 0}}));
9  // Van-e olyan k hosszú, aminek a végére lehet tenni a p-t
10 // ha van akkor id-t ad vissza.
11 int extends(int k, pont p) {
12   auto it = best[k].lower_bound({p.x, -1});
13   if (it == best[k].begin()) return -1;
14   return (--it)->y < p.y ? it->id : -1;
15 }
16 // Mi a leghosszabb, aminek a végére lehet tenni a p-t
17 int search_longest_extendable(pont p) {
18   int l = 0, r = best.size();
19   while (l + 1 < r) {
20     int mid = (l + r) / 2;
21     if (extends(mid, p) >= 0) l = mid;
22     else r = mid;
23   }
24   return l;
25 }
```

Hadjárat - Kód

```
1  // A best[k] -ba beszúrjuk p-t
2  void improve(int k, pont p) {
3    if (k >= best.size()) {
4      best.push_back(set<pont>({p}));
5      return;
6    }
7    auto it = best[k].lower_bound(p);
8    if (it != best[k].begin()) {
9      auto prev = it;
10     --prev;
11     if (prev->x == p.x || prev->y == p.y) {
12       // van a p-nél jobb már bent
13       return;
14     }
15   }
16   it = best[k].insert(it, p);
17   ++it;
18   while (it != best[k].end() && it->y >= p.y) {
19     it = best[k].erase(it);
20   }
21 }
```

Hadjárat - Kód

```
1  int main() {
2    int n; cin >> n;
3    vector<pont> v(n + 1);
4    for (int i = 1; i <= n; i++) {
5      cin >> v[i].x >> v[i].y; v[i].id = i;
6    }
7    v[0] = {-1, -1, 0};
8    vector<int> prev(n + 1);
9    for (int i = 1; i <= n; i++) {
10     int k = search_longest_extendable(v[i]);
11     prev[i] = extends(k, v[i]);
12     improve(k + 1, v[i]);
13   }
14   // Rekonstrukcio
15   int m = best.back().begin()->id;
16   vector<int> sol;
17   while (m != 0) {
18     sol.push_back(m);
19     m = prev[m];
20   }
21   reverse(sol.begin(), sol.end());
22   cout << best.size() - 1 << endl;
23   for (int x : sol) cout << x << " ";
24 }
```

LIS of Sequence (Codeforces)

Adott egy sorozat:

$$a_1, a_2, \ldots, a_n \qquad (1 \leq n \leq 100\,000)$$

Egy növekvő részsorozat:

$$a_{i_1} < a_{i_2} < \cdots < a_{i_k}, \quad i_1 < i_2 < \cdots < i_k$$

Tekintsük az összes a-beli leghosszabb növekvő részsorozatot (LIS).

Feladat: Minden i indexet három csoport valamelyikébe kell sorolnunk:

- **1**: a_i nem szerepel egyetlen LIS-ben sem
- **2**: a_i szerepel legalább egy LIS-ben, de nem mindegyikben
- **3**: a_i szerepel minden LIS-ben

Előfeldolgozás — LIS előre és visszafelé

Két klasszikus LIS-számítást végzünk $O(N \log N)$ időben.

Előre:

- `pref[i]` = LIS hossza, ami az `i`-edik elemben végződik

Visszafelé:

- `suff[i]` = LIS hossza, ami az `i`-edik elemből indul
- (technikai trükk: visszafelé haladás + értékek ellentettre állítása)

A LIS hossza:

$$L = \max_i \mathtt{pref[i]}$$

Mikor lehet egy elem LIS része?

Egy kulcsmegfigyelés:

Az `i`*-edik elem akkor és csak akkor lehet része egy LIS-nek, ha*

$$\texttt{pref[i]} + \texttt{suff[i]} - 1 = \texttt{L}$$

Értelmezés:

- `pref[i]`: mennyi elem lehet `i`-ig
- `suff[i]`: mennyi elem lehet `i`-től kezdve
- -1: az a_i-t ne számoljuk kétszer

Következmény:

- ha nem teljesül $\Rightarrow$ **1-es csoport**
- ha teljesül $\Rightarrow$ benne van *legalább egy* LIS-ben

Mikor szerepel egy elem minden LIS-ben?

Gondolkodjunk a LIS-ekről „szintek” szerint.

- $\texttt{pref[i]} = \texttt{k}$ jelentése: az a_i elem a LIS $\texttt{k}$**-adik pozícióján** állhat

Fontos tulajdonság:

- Ha a_i valamely LIS $\texttt{k}$-adik pozícióján áll, akkor minden LIS-ben, amiben szerepel, ugyanúgy a $\texttt{k}$-adik pozícióban lesz.
- Ha van olyan LIS, amiben nem szerepel a_i, akkor ott szükségképpen egy másik elem lesz a $\texttt{k}$-adik helyen.

Ezért:

- megszámoljuk minden $\texttt{k}$-ra, hányféle elem lehet a $\texttt{k}$-adik helyen,
- azaz $\texttt{cnt[k]} =$ hány olyan index van, amelyre

$$\texttt{pref[i]} = \texttt{k} \quad \text{és} \quad \texttt{pref[i]} + \texttt{suff[i]} - 1 = \texttt{L}$$

Az a_i elem akkor és csak akkor szerepel **minden** LIS-ben, ha:

- $\texttt{pref[i]} + \texttt{suff[i]} - 1 = \texttt{L}$ (része lehet LIS-nek)
- és az adott $\texttt{pref[i]}$ szinten **ő az egyetlen választható elem**

LIS of Sequence - Összefoglalás

A megoldás lépései:

- `pref[i]` és `suff[i]` kiszámítása $O(N \log N)$ időben
- $L = \max \mathtt{pref[i]}$
- Minden indexre eldöntjük, hogy $\mathtt{pref[i]} + \mathtt{suff[i]} - 1 = L$
  - Ha nem $\Rightarrow$ **1-es csoport**
- Szinteken megszámoljuk a választható elemeket
  - Ha > 1 elem lehet az ő szintjén $\Rightarrow$ **2-es csoport**
  - Ha csak 1 elem $\Rightarrow$ **3-as csoport**

Időkomplexitás: $O(N \log N)$ **Memória:** $O(N)$

A megoldás nem LIS-eket számol, hanem a LIS-ek szerkezetét elemzi.

LIS of Sequence - Kód

```
int n; cin >> n;
vector<int> a(n), pref(n), suff(n), cnt(n + 1), dp;
for (int i = 0; i < n; i++) {
  cin >> a[i];
  auto it = lower_bound(dp.begin(), dp.end(), a[i]);
  pref[i] = it - dp.begin() + 1;
  if (it != dp.end()) *it = a[i];
  else dp.push_back(a[i]);
  a[i] = -a[i];
}
int L = dp.size();
dp.clear();
for (int i = n - 1; i >= 0; i--) {
  auto it = lower_bound(dp.begin(), dp.end(), a[i]);
  suff[i] = it - dp.begin() + 1;
  if (pref[i] + suff[i] - 1 == L) cnt[pref[i]]++;
  if (it != dp.end()) *it = a[i];
  else dp.emplace_back(a[i]);
}
for (int i = 0; i < n; i++) {
  if (pref[i] + suff[i] - 1 == L) {
    if (cnt[pref[i]] == 1) cout << "3";
    else cout << "2";
  } else cout << "1";
```

Increasing Subsequence II (CSES)

Adott egy n hosszú sorozat:

$$x_1, x_2, \ldots, x_n \qquad (1 \leq n \leq 2 \cdot 10^5)$$

Egy részsorozat **növekvő**, ha

$$x_{i_1} < x_{i_2} < \cdots < x_{i_k}.$$

Feladat:

- Számoljuk meg az *összes* növekvő részsorozatot!
- Két részsorozat különböző, ha más indexeket használ.
- Az eredményt $10^9 + 7$-tel modulo kérjük.

Példa:

$$[2, 1, 3] \Rightarrow 5 \text{ darab: } [2], [1], [3], [2, 3], [1, 3]$$

Increasing Subsequence II - DP megfogalmazás

Legyen:

$$\texttt{dp[i]} = \text{az } \texttt{x}_\texttt{i}\text{-ben végződő növekvő részsorozatok } \textbf{száma}.$$

Ekkor:

$$\texttt{dp[i]} = 1 + \sum_{\texttt{j<i},\ \texttt{x}_\texttt{j}<\texttt{x}_\texttt{i}} \texttt{dp[j]}$$

- Az 1 az egyelemű részsorozat: $[\texttt{x}_\texttt{i}]$
- Minden kisebb korábbi elemre rá tudjuk fűzni

Naiv megoldás:

- dupla ciklus $\rightarrow \texttt{O}(\texttt{n}^2)$
- ennél hatékonyabb kell

Increasing Subsequence II - DP Optimalizálás

Az átmenet lényege:

$$\sum_{x_j < x_i} dp[j]$$

Ez egy **prefix összeg** a „érték szerint” rendezett elemekre.

Kulcsötlet:

- Koordinátakompresszió az x_i-kre
- Fenwick / szegmensfa:
  - index: érték
  - tárolt érték: dp-összeg

Minden lépésben az aktuális x_i elemre:

1. lekérdezzük: $S = \sum_{v < x_i} dp[v]$
2. $dp[i] = 1 + S$
3. frissítjük az x_i pozícióját

Komplexitás:

$$O(n \log n)$$

Alkalmazások

Sorozat rendezése minimális mozgatással

Adott egy sorozat:

$$a_1, a_2, \ldots, a_n$$

Egy lépésben:

- kiválasztunk egy elemet,
- eltávolítjuk,
- és tetszőleges másik helyre visszaszúrjuk.

Cél:

- a sorozat legyen **növekvő (nem csökkenő)**,
- a lépések száma legyen minimális.

Kulcsmegfigyelés

Ha egy elem **nem kerül nozgatásra**, akkor:

- megmarad az eredeti helyén,
- és relatív sorrendje a többi megmaradó elemmel változatlan.

Ezért a megmaradó elemek:

- egy **növekvő részsorozatot** alkotnak.

Megfordítva:

- bármely növekvő részsorozatot meghagyhatunk,
- a többi elemet kivágjuk és megfelelő helyre beszúrjuk.

Következtetés:

$$\text{max. megmaradó elemek} = \text{LIS hossza}$$

Szemléltetés: LIS = megtartott elemek

Kezdeti sorrend:

2	7	3	8	4	6	9	5

Szemléltetés: LIS = megtartott elemek

Kezdeti sorrend:

2	7	3	8	4	6	9	5

Megtartott (LIS) és kivágott elemek:

2	7	3	8	4	6	9	5

Szemléltetés: LIS = megtartott elemek

Kezdeti sorrend:

2	7	3	8	4	6	9	5

Megtartott (LIS) és kivágott elemek:

2	7	3	8	4	6	9	5

Végleges sorrend a mozgatások után:

2	3	4	5	6	7	8	9

Optimalitás bizonyítása

Legyen $\mathtt{L}$ a leghosszabb növekvő részsorozat hossza.

Alsó korlát:

- Legfeljebb $\mathtt{L}$ elemet hagyhatunk meg,
- különben nem lennének növekvők.

Felső korlát:

- Egy $\mathtt{L}$ hosszú LIS-t meghagyva
- a maradék $\mathtt{n} - \mathtt{L}$ elemet mindig vissza tudjuk szúrni megfelelő helyre.

Tehát:

$$\boxed{\text{minimális lépésszám} = \mathtt{n} - \text{LIS hossza}}$$

LCS on Permutations (USACO Guide)

Két permutációt kapunk a és b formában, mindkettő hossza N.

- $a_1, a_2, \ldots, a_N$ az $\{1, \ldots, N\}$ egy permutációja
- $b_1, b_2, \ldots, b_N$ szintén az $\{1, \ldots, N\}$ egy permutációja

Feladat:

- Határozzuk meg a leghosszabb közös részsorozat (LCS) hosszát!

Korlátok:

- $1 \leq N \leq 100\,000$

Példa:

$$a = [4, 1, 7, 3, 6, 2, 5]$$

$$b = [1, 3, 4, 2, 7, 6, 5]$$

esetén az LCS hossza 4, például az 1, 3, 6, 5 számok egy 4 hosszú közös részsorozatot alkotank.

Megfigyelés

A klasszikus LCS dinamikus programozás:

$$\texttt{dp[i][j]} = \texttt{LCS(a[1..i], b[1..j])} \text{ hossza.}$$

Probléma:

- $\mathcal{O}(\texttt{N}^2)$ idő – túl lassú

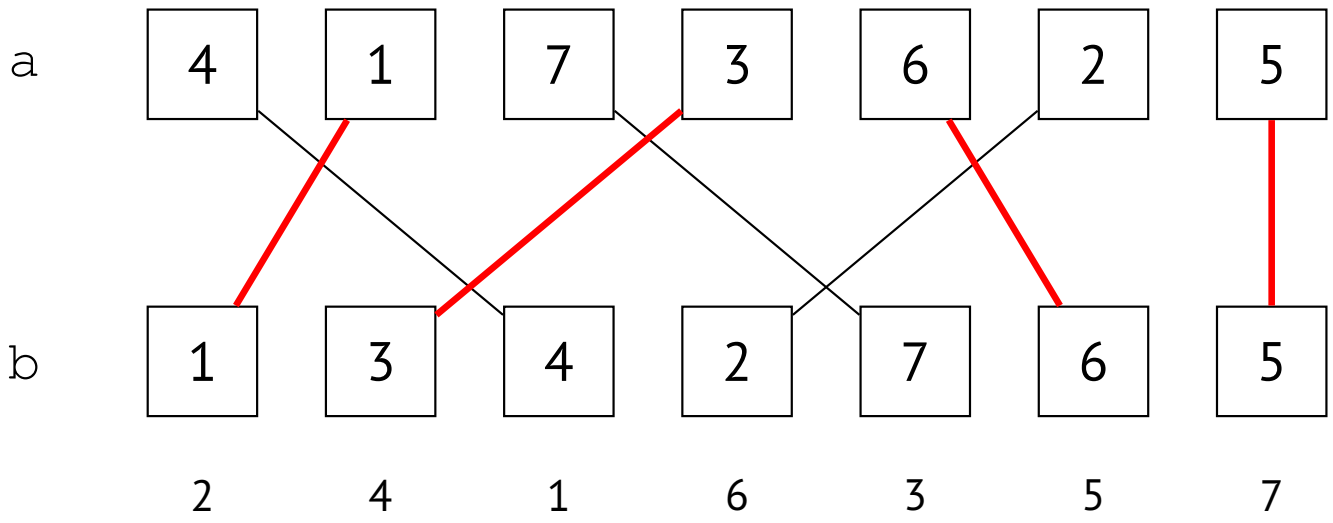
Kulcsmegfigyelés:

- `a` és `b` **permutációk**
- minden szám pontosan egyszer szerepel

  Egy elemhez pontosan egy illeszkedő pozíció tartozik a másik sorozatban.

LCS → LIS vizuális átalakítás

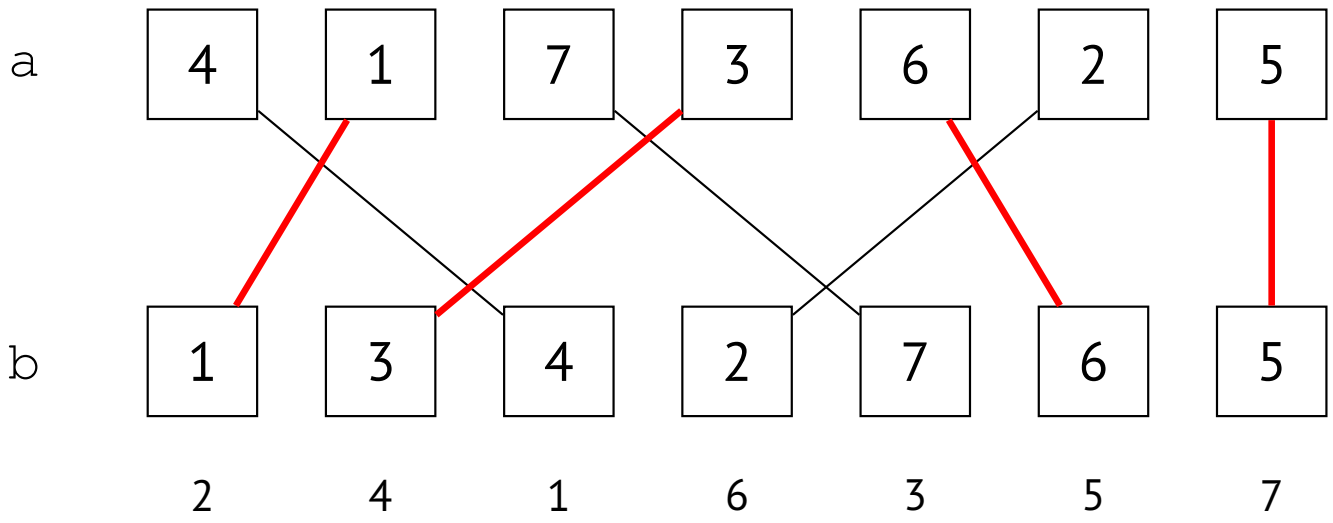
Írjuk egymás alá `a`-t és `b`-t, és kössük össze az azonos elemeket. Akkor alkothatnak közös részsorozatot az elemek, ha nem metszik egymást a hozzájuk tartozó vonalak.



Írjuk a `b` elemei alá, hogy hányadik pozíción vannak az `a`-ban. Ezeket a számokat tekintve mi a feltétele annak, hogy a vonalak nem metszik egymást?

LCS → LIS vizuális átalakítás

Írjuk egymás alá `a`-t és `b`-t, és kössük össze az azonos elemeket. Akkor alkothatnak közös részsorozatot az elemek, ha nem metszik egymást a hozzájuk tartozó vonalak.



Írjuk a `b` elemei alá, hogy hányadik pozíción vannak az `a`-ban. Ezeket a számokat tekintve mi a feltétele annak, hogy a vonalak nem metszik egymást?

*A `b` alá írt indexek sorozatán már **LIS**-t keresünk.*

Algoritmus – példán

$$a = [4, 1, 7, 3, 6, 2, 5] \quad b = [1, 3, 4, 2, 7, 6, 5]$$

1. Számítsuk ki az a inverz permutációját:

$$pos[x] = \text{x pozíciója } a\text{-ban}$$

x	1	2	3	4	5	6	7
pos[x]	2	6	4	1	7	5	3

2. Képezzük az új c sorozatot:

$$c[i] = pos[b[i]]$$

$$c = [2, 4, 1, 6, 3, 5, 7]$$

3. c leghosszabb növekvő részsorozatát keressük.

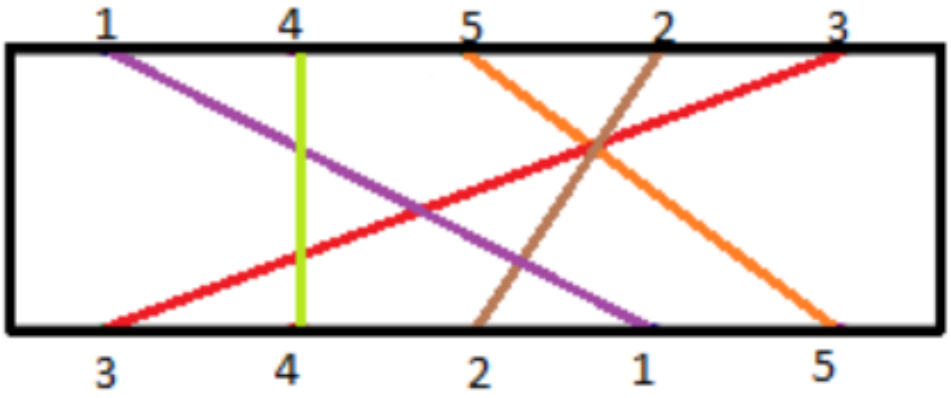
$$LIS_c = [2, 4, 5, 7]$$

Komplexitás: Idő: $\mathcal{O}(N \log N)$ Memória: $\mathcal{O}(N)$

Így a permutációkon vett LCS hatékonyan megoldható.

Optical Experiment (Codeforces)

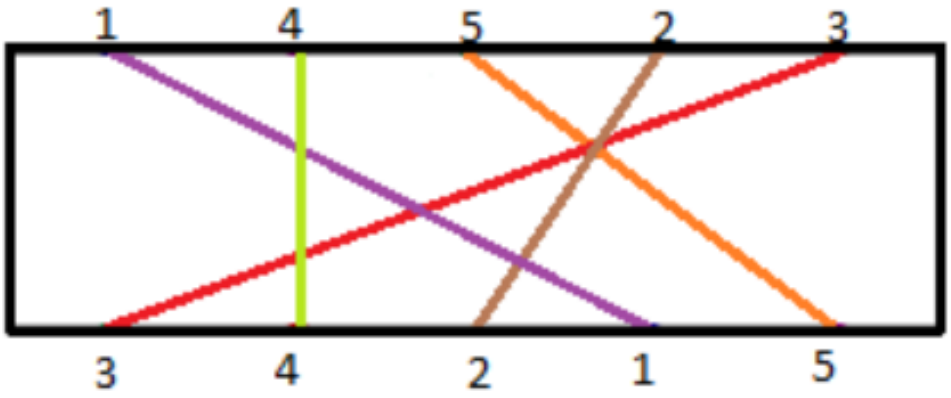
`https://codeforces.com/contest/67/problem/D`



Adott `n` sugár, amelyek mind a felső `n` nyílás egyikéből az alsó `n` nyílás egyikébe tartanak. Válasszuk ki a lehető legtöbb sugarat úgy, hogy azok mind messék egymást.
Például a fenti ábrára a megoldás 3: az `5, 2, 3` azonosítójú sugarakat választva.

Optical Experiment (Codeforces)

https://codeforces.com/contest/67/problem/D



Adott n sugár, amelyek mind a felső n nyílás egyikéből az alsó n nyílás egyikébe tartanak. Válasszuk ki a lehető legtöbb sugarat úgy, hogy azok mind messék egymást.
Például a fenti ábrára a megoldás 3: az 5, 2, 3 azonosítójú sugarakat választva.

Megoldás:

- Tekintsük azt a c sorozatot, ahol c_i = a felső i. nyílásból induló sugár hányadik alsó nyílásba megy.
- Az egymást metsző sugarak c-ben LDS-t (leghosszabb csökkenő részsorozatot) alkotnak.

Sum Partition (Kódkupa döntő 2024)

https://njudge.hu/problemset/main/KK24_sumpartition/

Adott két tömb:

$$\mathtt{A}_1, \mathtt{A}_2, \ldots, \mathtt{A}_\mathtt{N} \quad (-10^{12} \leq \mathtt{A}_\mathtt{i} \leq 10^{12}),$$

$$\text{és } \mathtt{B}_1, \mathtt{B}_2, \ldots, \mathtt{B}_\mathtt{M} \quad (1 \leq \mathtt{B}_\mathtt{i} \leq 10^{6}),$$

$$\text{ahol } 1 \leq \mathtt{N}, \mathtt{M} \leq 200\,000.$$

Cél: osszuk fel A-t és B-t is K résztömbre úgy, hogy az összegük rendre megegyezzen.

Példa:

$$\mathtt{A} = [1, 3, 6, -3, 9], \quad \mathtt{B} = [1, 9, 2, 4], \quad \mathtt{K} = 3$$

esetén ezt el tudjuk érni úgy, hogy

- A felosztása: [1], [3, 6], [−3, 9]
- B felosztása: [1], [9], [2, 4]

Sum Partition - Megfigyelések

Vegyük `A` és `B` prefix összegét:

$$\texttt{prefA} = [1, 4, 10, 7, 16]$$

$$\texttt{prefB} = [1, 10, 12, 16]$$

Ezekben egy `K` hosszú közös részsorozatot keresünk, ami az utolsó elemben végződik mindkettőben.

- Ha az utolsó elem nem egyezik, akkor triviálisan nincs megoldás.
- Ha van `K`-nál hosszabb közös részsorozat, akkor `K` hosszú is van, hiszen kihagyhatunk közbülső elemeket
- Tehát átfogalmazhatjuk a feladatot úgy, hogy a két prefix összeg tömb leghosszabb közös részsorozatát (LCS) keressük.

Első megközelítés:

- Az LCS-re ismert $O(N \cdot M)$ megoldással részpontokat lehet szerezni, de $N, M = 200\,000$ esetén időlimites
- Hatékonyabb megoldásra akkor van esélyünk, ha `prefA` vagy `prefB` valamilyen speciális tulajdonságát kihasználjuk. Következik-e valami a korlátokból?

Sum Partition - Megoldás

Kulcs észrevétel:

- Mivel $1 \leq \mathtt{B_i}$, ezért `prefB` szigorúan monoton nő
- Így `prefA` bármely eleméhez legfeljebb egy vele egyező van `prefB`-ben
- Ez elég ahhoz, hogy az LCS on Permutations feladat megoldása működjön

Megoldás:

- Legyen

$$\mathtt{C_i} = \begin{cases} \mathtt{j}, & \text{ha van olyan } \mathtt{j}\text{, melyre } \mathtt{prefA_i} = \mathtt{prefB_j} \\ -1, & \text{egyébként} \end{cases}$$

- `C` sorozat nemnegatív elemeiben LIS-t keresünk. Ez megfelel `LCS(prefA, prefB)`-nek.
- `C` előállításához használhatunk map-et, vagy bináris keresést is `prefB` elemein.

Komplexitás: $\mathtt{O(M + N \cdot (\log M + \log N)}$

Feladatok

Önálló munkára

Consecutive Subsequence – Feladat

https://codeforces.com/contest/977/problem/F

Adott egy n hosszú egész tömb $a_1, a_2, \ldots, a_n$.

Egy olyan részsorozatot kell kiválasztani maximális hosszal, amely **szigorúan növekvő, egymást követő egész számokból áll**, azaz

$$[x, x+1, \ldots, x+k-1]$$

alakú valamely x és k esetén. (A részsorozat elemei az eredeti sorrendet megtartják, de nem kell szomszédosnak lenniük.)

Bemenet:

- Első sor: n $(1 \leq n \leq 2 \cdot 10^5)$
- Második sor: $a_1, a_2, \ldots, a_n$ $(1 \leq a_i \leq 10^9)$

Kimenet:

- Első sor: k – a maximális hossz
- Második sor: egy maximális hosszú részsorozat indexei

Példa bemenet:

7
3 3 4 7 5 6 8

Kimenet:

4
2 3 5 6

Global Warming – Feladat

CEOI 2018, Day 1
`https://oj.uz/problem/view/CEOI18_glo`

Johnny történelmi hőmérsékleti adatokat vizsgál n egymást követő napra, ahol a i-edik napon mért hőmérséklet t_i volt.

Célja egy lehető leghosszabb **szigorúan növekvő részsorozat** (LIS) megtalálása a

$$(t_1, t_2, \ldots, t_n)$$

sorozatban.

Johnny azonban egy kicsit csalhat:

- kiválaszt egy **nem üres intervallumot** $[l, r]$,
- választ egy d egész számot, ahol $-x \leq d \leq x$,
- és minden $i \in [l, r]$ esetén $t_i \leftarrow t_i + d$.

Megengedett $d = 0$ is.

Feladat: Határozd meg a lehető legnagyobb LIS hosszát egyetlen ilyen módosítás után.

Global Warming – Feladat

https://oj.uz/problem/view/CEOI18_glo

Bemenet:

- Első sor: $n, x \quad (1 \leq n \leq 2 \cdot 10^5,\ 0 \leq x \leq 10^9)$
- Második sor: $t_1, t_2, \ldots, t_n \quad (1 \leq t_i \leq 10^9)$

Kimenet:

- Egy egész szám – a maximálisan elérhető LIS hossz

Példa bemenet:

```
8 10
7 3 5 12 2 7 3 4
```

Kimenet:

```
5
```

Candy Machine – Feladat

Baltic Olympiad in Informatics 2009
`https://boi.cses.fi/files/boi2009_day1.pdf`
Beadni itt lehet: `https://cses.fi/107/list/`

Egy cukorgyárban működik egy különleges gép, amely édességeket gyárt. A gépnek n darab kimeneti rekesze van, számozva 1-től n-ig. Minden cukorka egy adott időpontban és egy adott rekeszből esik ki.

A gyártás megkezdése előtt ismert minden cukorkára:

- s_i – melyik rekeszből esik ki,
- t_i – mikor esik ki.

Minden (s_i, t_i) pár egyedi.

A cukorkák elkapására automata kocsik helyezhetők el a rekeszek alatt.

- Egy kocsi sebessége: **1 rekesz / másodperc**,
- A gyártás kezdete előtt tetszőleges kezdőrekeszre beállítható,
- Egy adott időpillanatban pontosan egy rekesz alatt lehet.

Feladat: Határozd meg a minimálisan szükséges kocsik számát úgy, hogy minden cukorka el legyen kapva, és add meg, melyik kocsi melyik cukorkát kapja el.

Candy Machine – Feladat

`https://boi.cses.fi/files/boi2009_day1.pdf`
Beadni itt lehet: `https://cses.fi/107/list/`

Bemenet:

- Első sor: n $(1 \leq n \leq 100\,000)$
- Következő n sor: s_i, t_i $(0 \leq s_i, t_i \leq 10^9)$

Kimenet:

- Első sor: w – a minimális kocsiszám
- Következő n sor: s_j, t_j, $w(j)$ – melyik kocsi kapja el az adott cukorkát

Példa bemenet:

```
5
1 1
2 3
1 5
3 4
2 6
```

Kimenet:

```
2
1 1 1
2 3 1
1 5 2
3 4 1
2 6 2
```

Lekötő feladatok

Zip Line
https://codeforces.com/contest/650/problem/D

CERC 2025 F. Foxes
https://assets.ucup.ac/statements/statements-4-10.pdf